

Royal Estate App

Complete Project Explanation in Roman English - Separate PDF for Client, api, Client/api, and AI folders

Is PDF me project ke tech stack, important functions, React hooks, events, backend routes, database flow, AI flow, image upload, authentication, search, messaging aur Home page Swiper fix sab simple Roman English me explain kiya gaya hai.

1. Project Overview

Royal Estate App ek full stack real estate platform hai. User property browse/search kar sakta hai, listing detail dekh sakta hai, account bana sakta hai, login kar sakta hai, listing create/update/delete kar sakta hai, owner ko message bhej sakta hai, aur AI advisor se real estate ya ROI related help le sakta hai.

- **Client/** - React frontend jo browser me UI dikhata hai.
- **api/** - Node.js Express backend jo MongoDB, auth, listings, upload, messages handle karta hai.
- **AI/** - Python FastAPI server jo AI chat, listing description generation, aur ROI prediction deta hai.
- **Client/api/** - Vercel serverless helper, jaise AI server keep-alive endpoint.

2. Tech Stack Used

Area	Tech	Use
Frontend	React + Vite	Fast SPA UI banane ke liye. Vite dev/build fast karta hai.
Styling	Tailwind CSS	Utility classes se responsive design aur dark royal theme.
Routing	react-router-dom	Pages ke beech navigation: Home, Search, Listing, Profile, SignIn.
State	Redux Toolkit + redux-persist	Logged-in user ko global state me rakhna aur refresh ke baad preserve karna.
API Calls	Axios	Frontend se backend REST APIs call karna.
i18n	i18next + react-i18next	English/Arabic translation aur RTL direction.
Carousel	Swiper	Home aur listing image slider.
SEO	react-helmet-async	Page title/meta description set karna.
Backend	Node.js + Express	REST API server.
Database	MongoDB + Mongoose	Users, listings, messages store karna.
Auth	JWT + cookie-parser + bcryptjs	Secure login, token cookie, password hashing.
Images	Cloudinary + multer	Images upload aur CDN optimized delivery.
AI	FastAPI + Cerebras SDK	AI chat, description generator, ROI report.

3. Frontend Flow - Client Folder

3.1 main.jsx

main.jsx app ka entry point hai. Yahi React ko HTML ke root element me render karta hai.

- **Provider** Redux store ko poori app me available banata hai.
- **PersistGate** redux-persist ko wait karata hai taaki saved user state load ho jaye.
- **import ./i18n** translation system initialize karta hai.

3.2 App.jsx

App.jsx main routing file hai. Isme BrowserRouter, Header, Routes aur PrivateRoute ka setup hai. useEffect language change par body direction set karta hai: Arabic me rtl, English me ltr.

```
useEffect(() => { document.body.dir = i18n.language === 'ar' ? 'rtl' : 'ltr'; }, [i18n.language]);
```

3.3 Home.jsx

Home page pe hero text, Swiper image carousel, recent offers, recent rent, recent sale, aur AIChatbot load hota hai.

- **useState** offerListings, rentListings, saleListings, loading, swiperKey store karta hai.
- **useEffect** first render par /api/listing/home?limit=4 call karta hai.
- **axiosInstance.get** backend se data fetch karta hai.
- **Swiper** images ko auto-slide + navigation + pagination ke saath show karta hai.
- **Link** slide click par user ko /listing/:id page par le jata hai.
- **Helmet** SEO title/meta description set karta hai.

Swiper fix: Carousel ab offer + rent + sale listings me se image available listings leta hai. Image w-screen container, h-full image, object-cover, responsive srcSet aur sizes=100vw ke saath full screen width tak cover karti hai.

3.4 Search.jsx

Search page filters ko URL query params me convert karta hai. Isse refresh ke baad bhi same filters stay karte hain.

- **sidebardata state** searchTerm, type, parking, furnished, offer, sort, order rakhta hai.
- **useEffect location.search** URL change hone par backend se filtered listings fetch karta hai.
- **Debounce useEffect** typing ke 500ms baad search URL update karta hai. Isse har key press par API hit nahi hoti.
- **handleChange** checkbox/input/select change handle karta hai.
- **handleSubmit** form submit par navigate karta hai.
- **onShowMoreClick** pagination ke liye startIndex set karke next listings load karta hai.

3.5 Listing Detail Page

Listing page listingId URL se leta hai, backend se listing fetch karta hai, images Swiper me dikhata hai, property overview, price, amenities, contact button aur AI investment report show karta hai.

- **useParams** URL se listingId nikalta hai.
- **useSelector** Redux se currentUser leta hai.
- **getAIAnalysis** AI FastAPI server ko property data bhejta hai aur ROI report lata hai.
- **onClick events** contact/AI report actions trigger karte hain.

3.6 CreateListing.jsx

Create listing form me user property details, price, currency, offer, amenities aur images add karta hai.

- **formData state** listing ke fields store karta hai.
- **files state** selected image files rakhta hai.
- **handleAiGenerate** AI server se property description generate karata hai.
- **storeImage** Cloudinary ko single image upload karta hai.
- **uploadInBatches** desktop par images ko batches me upload karta hai for stability.
- **handleImageSubmit** selected files validate + upload karta hai.

- **handleRemoveImage** preview list se image remove karta hai.
- **handleSubmit** validation ke baad /api/listing/create par POST karta hai.

3.7 Redux Files

store.js Redux store configure karta hai. redux-persist localStorage me user state save karta hai. userSlice.js signInStart, signInSuccess, signInFailure, updateUser, deleteUser, signOut reducers define karta hai. Reducer state ko predictable tarike se update karta hai.

3.8 Components

- **Header** logo, search box, language switcher, profile/signin link show karta hai.
- **PrivateRoute** currentUser nahi hai to protected pages ko sign-in par redirect karta hai.
- **ListingItem** listing card reusable component hai.
- **Contact** owner ko message bhejne ka UI hai.
- **Inbox** user messages display karta hai.
- **AIChatbot** floating AI advisor chat widget hai. Messages state, typing state, input state use karta hai.

4. React Hooks and Events - Simple Explanation

4.1 useState

useState component ke andar changing data store karta hai. Jab state update hoti hai to component re-render hota hai. Project examples: loading, listings, formData, input, messages, isOpen, copied, contact.

4.2 useEffect

useEffect side effects ke liye hota hai: API call, language direction change, scroll update, URL params sync, debounce timer.

- **Empty dependency []** means effect sirf first render ke baad chalega.
- **[!18n.language]** means language change par effect dobara chalega.
- **[location.search]** means URL search query change par listings refetch.
- **Cleanup return** debounce me timeout clear karta hai.

4.3 useSelector

useSelector Redux store se data read karta hai. Project me currentUser get karne ke liye use hua hai.

4.4 useNavigate, useParams, Link

- **useNavigate** programmatically route change karta hai.
- **useParams** URL dynamic param leta hai. Example: /listing/:listingId.
- **Link** SPA navigation karta hai bina full page reload ke.

4.5 Common Events

- **onChange** input/checkbox/select value change par state update karta hai.
- **onSubmit** form submit par API call ya navigation trigger karta hai.
- **onClick** buttons ke actions: language toggle, AI generate, upload, remove image, show more.

- **input trim logic** AIChatbot empty message prevent karta hai.

5. Backend Flow - api Folder

5.1 index.js

Backend ka entry file Express app banata hai. dotenv env variables load karta hai, MongoDB connect karta hai, Cloudinary configure karta hai, CORS set karta hai, JSON parser aur cookie parser enable karta hai, routes mount karta hai.

```
app.use('/api/auth', authRouter); app.use('/api/listing', listingRouter); app.use('/api/message', messageRouter);
```

5.2 Auth Flow

- **signup** password ko bcryptjs se hash karke user save karta hai.
- **signin** email se user find karta hai, password compare karta hai, JWT banata hai, httpOnly cookie me set karta hai.
- **google** existing Google email user ko login karata hai, nahi to new user create karta hai.
- **signOut** access_token cookie clear karta hai.

5.3 verifyToken Middleware

verifyToken protected routes ka gatekeeper hai. Cookie se access_token leta hai, JWT verify karta hai, valid ho to req.user me user id attach karta hai. Invalid token par Unauthorized/Forbidden error deta hai.

5.4 Listing Routes and Controllers

- **POST /api/listing/create** logged-in user ki listing create karta hai.
- **DELETE /api/listing/delete/:id** owner-only delete.
- **POST /api/listing/update/:id** owner-only update.
- **GET /api/listing/get/:id** single listing detail.
- **GET /api/listing/get** search, filters, sort, pagination.
- **GET /api/listing/home** homepage ke liye fast endpoint: offers, rent, sale parallel queries.
- **GET /api/listing/get-all-chatbot** chatbot ke context ke liye listings.

Performance point: getListings empty searchTerm par regex nahi chalata. Ye important hai kyunki empty regex wide scan kara sakta hai aur API slow ho sakti hai.

5.5 Mongoose Models

- **User model** account fields store karta hai.
- **Listing model** property name, description, address, price, amenities, type, offer, imageUrls, userRef store karta hai.
- **Message model** senderId, receiverId, listingId, text store karta hai.
- **Indexes** createdAt, type+createdAt, offer+createdAt queries ko fast banate hain.

5.6 Upload and Cloudinary

Images Cloudinary par upload hoti hain. Cloudinary CDN images fast serve karta hai. Frontend image util Cloudinary URL me width, crop, quality auto, format auto transformation add karta hai.

5.7 Message Flow

Contact component message send karta hai. Backend sendMessage req.user.id ko senderId banata hai aur receiverId/listingId/text save karta hai. Inbox getMessages receiverId ke basis par messages laata hai.

6. Client/api Folder

Client/api/keep-alive.js ek Vercel-style serverless handler hai. Ye AI server ke /docs endpoint ko ping karta hai. Purpose: deployed AI service ko warm/awake rakhna ya health check karna. Success par { ok: true }, failure par { ok: false } return karta hai.

7. AI Folder Flow

AI folder me FastAPI server hai. Ye Node backend se separate service hai. Frontend directly AI server endpoints ko call karta hai.

- **GET / health/root** message return karta hai.
- **POST /chat** user ke message ka language detect karta hai aur Cerebras LLM se short advisor response laata hai.
- **POST /generate-listing-ai** title, location, features se professional listing description generate karta hai.
- **POST /ai-roi-prediction** property title, location, price, features ke basis par ROI/rental yield/appreciation recommendation generate karta hai.
- **get_live_rates** exchange API se rates la sakta hai; failure par fallback rates use karta hai.
- **CORS** frontend domains ko allow karta hai.

8. End-to-End User Flow

8.1 Home Page Flow

User Home open karta hai -> React Home component mount hota hai -> useEffect backend /api/listing/home?limit=4 call karta hai -> backend MongoDB se offers/rent/sale parallel fetch karta hai -> frontend state update hoti hai -> Swiper hero carousel aur listing cards render hote hain.

8.2 Search Flow

User search box ya filters change karta hai -> handleChange state update karta hai -> debounce useEffect 500ms baad URL query update karta hai -> location.search change hota hai -> fetchListings API call karta hai -> backend filters build karta hai -> MongoDB result return -> frontend cards update.

8.3 Login Flow

User email/password submit karta hai -> frontend signin API call karta hai -> backend user find + password compare karta hai -> JWT create karta hai -> httpOnly cookie set karta hai -> user data Redux me store hota hai -> PrivateRoute pages accessible ho jate hain.

8.4 Create Listing Flow

User form fill karta hai -> images select karta hai -> Cloudinary upload hota hai -> secure URLs formData.imageUrls me add hoti hain -> submit par backend create route call hota hai -> verifyToken user identify karta hai -> MongoDB listing save -> user listing detail page par navigate hota hai.

8.5 AI ROI Flow

Listing detail page par Generate AI Investment Report click -> getAIAnalysis listing data AI server ko POST karta hai -> FastAPI Cerebras model ko prompt bhejta hai -> ROI analysis return hota hai -> React state analysis set hoti hai -> report UI me show hoti hai.

9. Important Fix Done in Home.jsx

- Swiper modules me Pagination add hua.
- Swiper CSS pagination import hua.
- Hero slider ke liye heroListings banaya gaya jo offer + rent + sale listings me se images available items leta hai.
- Carousel container w-screen and responsive height use karta hai: mobile 420px, desktop 550px.
- Image class block w-full h-full object-cover hai, isse image full width cover karti hai.
- Cloudinary optimized image URL + responsive srcSet use hua, sizes=100vw set hua.
- Loop tabhi active hota hai jab more than one slide ho.

10. Common Terms Easy Meaning

- **Component** React ka reusable UI block, jaise Header, ListingItem.
- **State** changing data, jaise form input, loading, listings.
- **Props** parent se child component ko bheja gaya data.
- **API** frontend aur backend ke beech data exchange ka endpoint.
- **Middleware** request aur response ke beech ka checking/helper function, jaise verifyToken.
- **Controller** route ka actual logic, jaise createListing.
- **Model** database document ka structure, jaise Listing schema.
- **JWT** signed token jo user identity prove karta hai.
- **httpOnly cookie** secure cookie jise JavaScript directly read nahi kar sakti.
- **CORS** browser rule jisse allowed frontend domains backend call kar sakte hain.
- **CDN** global image/static delivery network.
- **Debounce** action delay karna taaki unnecessary API calls kam ho.

11. Short Conclusion

Royal Estate App me frontend React SPA, backend Express REST API, MongoDB database, Cloudinary image pipeline, Redux auth state, i18n multilingual support, Swiper image carousel, messaging, aur separate FastAPI AI service integrated hai. Overall flow clean hai: frontend user action leta hai, API call karta hai, backend DB/AI/media service se data process karta hai, phir frontend state update karke UI refresh karta hai.